Міністерство освіти і науки України
Національний технічний університет
«Дніпровська політехніка»

ЗВІТ

про виконання лабораторної роботи № 1

з дисципліни
Поглиблене програмування в середовищі Java

тема роботи:
Розробка REST-застосунку на Spring Boot

Виконав(ла): ст. гр. 121-22-2
Голоцван Іван Миколайович

Прийняв: Мінєєв О.С.

Дніпро
2026

Тема: Розробка REST-застосунку на Spring Boot

Мета: Розробити повноцінний клієнт-серверний застосунок на мові Java, що реалізує обрану бізнес-логіку (наприклад, прототип вашої майбутньої дипломної роботи або сервіс для автоматизації власних задач). Сама логіка може бути будь яка. Жодних вимог. Уявіть що це ваш старт-ап

Хід роботи

В якості REST-застосунку було обрано створення программного забезпечення для відслідковування зміни ціни на парфуми

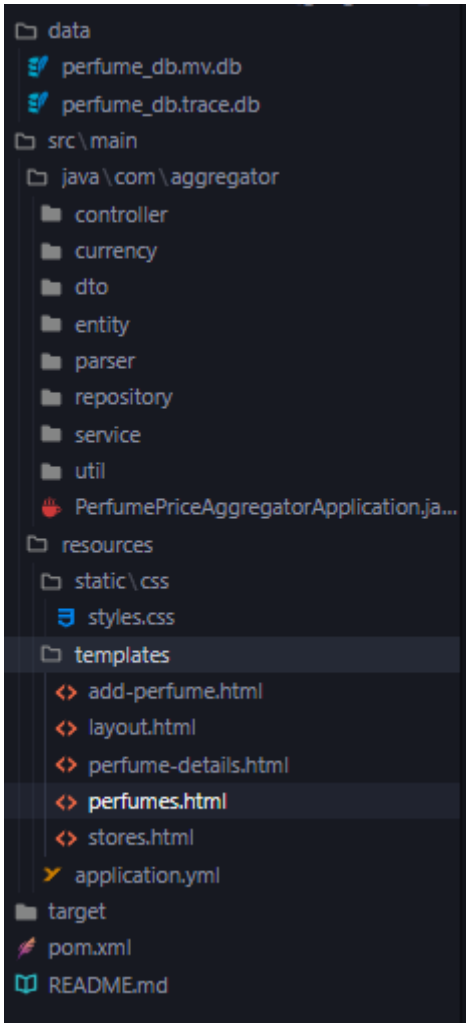
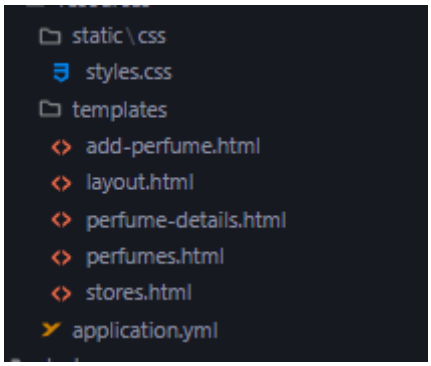


Рис. 1. Структура проєкту

/data - файли для тимчасовою бази данних

/src - файли для застосунку

/resources - конфігураційні та html/css файли



Style.css

```
:root {
    --bg-color: #1e1e1e;
    --panel-bg: #2d2d2d;
    --border-color: #404040;
    --text-main: #e0e0e0;
    --text-muted: #9e9e9e;
    --accent: #007acc;
    --accent-hover: #0098ff;
    --danger: #d16969;
    --success: #4caf50;

    --font-family: -apple-system, BlinkMacSystemFont, "Segoe
UI", Roboto, Helvetica, Arial, sans-serif;
}

body {
    margin: 0;
    padding: 0;
    background-color: var(--bg-color);
    color: var(--text-main);
    font-family: var(--font-family);
    font-size: 14px;
```

```
    display: flex;
    height: 100vh;
    overflow: hidden;
}

/* Sidebar */
.sidebar {
    width: 250px;
    background-color: var(--panel-bg);
    border-right: 1px solid var(--border-color);
    display: flex;
    flex-direction: column;
}

.sidebar-header {
    padding: 20px;
    font-size: 16px;
    font-weight: 600;
    border-bottom: 1px solid var(--border-color);
    letter-spacing: 0.5px;
}

.nav-links {
    list-style: none;
    padding: 0;
    margin: 0;
}

.nav-links li a {
    display: block;
    padding: 12px 20px;
```

```
    color: var(--text-muted);
    text-decoration: none;
    border-left: 3px solid transparent;
    transition: all 0.2s ease;
}

.nav-links li a:hover, .nav-links li a.active {
    color: var(--text-main);
    background-color: rgba(255, 255, 255, 0.05);
    border-left-color: var(--accent);
}

/* Main Content */
.main-content {
    flex: 1;
    display: flex;
    flex-direction: column;
    overflow-y: auto;
}

.topbar {
    height: 60px;
    border-bottom: 1px solid var(--border-color);
    display: flex;
    align-items: center;
    padding: 0 20px;
    background-color: var(--bg-color);
}

.page-title {
    font-size: 18px;
```

```
    font-weight: 500;
    margin: 0;
}

.content-area {
    padding: 24px;
    flex: 1;
}

/* Components */
.panel {
    background-color: var(--panel-bg);
    border: 1px solid var(--border-color);
    border-radius: 4px;
    padding: 20px;
    margin-bottom: 20px;
}

table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 10px;
}

th, td {
    padding: 12px 15px;
    text-align: left;
    border-bottom: 1px solid var(--border-color);
}

th {
```

```
    font-weight: 500;
    color: var(--text-muted);
    text-transform: uppercase;
    font-size: 12px;
    letter-spacing: 0.5px;
}

tr:hover td {
    background-color: rgba(255, 255, 255, 0.02);
}

.btn {
    background-color: var(--panel-bg);
    color: var(--text-main);
    border: 1px solid var(--border-color);
    padding: 8px 16px;
    cursor: pointer;
    font-family: var(--font-family);
    font-size: 13px;
    transition: all 0.2s;
    text-decoration: none;
    display: inline-block;
}

.btn:hover {
    background-color: var(--border-color);
}

.btn-primary {
    background-color: var(--accent);
    border-color: var(--accent);
```

```
    color: #fff;
}

.btn-primary:hover {
    background-color: var(--accent-hover);
    border-color: var(--accent-hover);
}

.btn-success {
    background-color: var(--success);
    border-color: var(--success);
    color: #fff;
}

.btn-sm {
    padding: 4px 8px;
    font-size: 12px;
}

.form-group {
    margin-bottom: 16px;
}

label {
    display: block;
    margin-bottom: 6px;
    color: var(--text-muted);
    font-size: 13px;
}

input[type="text"], input[type="url"], select {
```

```
    width: 100%;
    padding: 10px;
    background-color: var(--bg-color);
    border: 1px solid var(--border-color);
    color: var(--text-main);
    font-family: var(--font-family);
    box-sizing: border-box;
    border-radius: 2px;
}

input:focus, select:focus {
    outline: none;
    border-color: var(--accent);
}

.price-tag {
    font-family: 'Courier New', Courier, monospace;
    font-weight: bold;
    color: var(--success);
}

.empty-state {
    text-align: center;
    padding: 40px;
    color: var(--text-muted);
}

.d-flex {
    display: flex;
}
```

```
.justify-between {
    justify-content: space-between;
}

.align-center {
    align-items: center;
}

.gap-10 {
    gap: 10px;
}
```

Add-perfumes.html

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{layout :: head('Add Perfume -
PPA')}"></head>
<body>
    <div th:replace="~{layout :: sidebar}"></div>

    <div class="main-content">
        <div class="topbar">
            <h1 class="page-title">Add New Perfume</h1>
        </div>

        <div class="content-area">
            <div class="panel" style="max-width: 500px;">
                <form id="addPerfumeForm"
onsubmit="submitForm(event)">
                    <div class="form-group">
                        <label for="brand">Brand</label>
```

```
                        <input type="text" id="brand"
name="brand" required placeholder="e.g. Dior">
                    </div>

                    <div class="form-group">
                        <label for="name">Perfume Name</label>
                        <input type="text" id="name"
name="name" required placeholder="e.g. Sauvage">
                    </div>

                    <div class="form-group">
                        <label for="gender">Gender /
Category</label>
                        <select id="gender" name="gender">
                            <option value="Men">Men</option>
                            <option
value="Women">Women</option>
                            <option
value="Unisex">Unisex</option>
                        </select>
                    </div>

                    <div style="margin-top: 24px;">
                        <button type="submit" class="btn btn-
primary">Save Perfume</button>
                    </div>
                </form>
            </div>
        </div>
    </div>
```

```
    <script>
        function submitForm(e) {
            e.preventDefault();

            const data = {
                brand: document.getElementById('brand').value,
                name: document.getElementById('name').value,
                gender: document.getElementById('gender').value
            };

            fetch('/api/perfumes', {
                method: 'POST',
                headers: {
                    'Content-Type': 'application/json'
                },
                body: JSON.stringify(data)
            })
            .then(res => {
                if(res.ok) {
                    window.location.href = '/perfumes';
                } else {
                    alert('Error adding perfume.');
                }
            });
        }
    </script>
</body>
</html>
```

Layout.html

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head th:fragment="head(title)">
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title th:text="${title}">PPA</title>
    <link rel="stylesheet" th:href="@{/css/styles.css}">
</head>
<body>
    <div th:fragment="sidebar" class="sidebar">
        <div class="sidebar-header">PPA WORKSPACE</div>
        <ul class="nav-links">
            <li><a th:href="@{/perfumes}">Perfumes</a></li>
            <li><a th:href="@{/add-perfume}">Add
Perfume</a></li>
            <li><a th:href="@{/stores}">Stores</a></li>
        </ul>
    </div>
</body>
</html>
```

Perfume details.html

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{layout :: head(${perfume.name} + ' -
PPA')}"></head>
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
<body>
    <div th:replace="~{layout :: sidebar}"></div>
```

```
    <div class="main-content">
        <div class="topbar">
            <h1 class="page-title" th:text="${perfume.brand} +
' ' + ${perfume.name}">Perfume Details</h1>
            <div style="flex:1"></div>
            <span class="btn" style="pointer-events:none;
background:transparent; border:none;"
th:text="${perfume.gender}"></span>
        </div>

        <div class="content-area">

            <div class="panel d-flex justify-between align-
center">
                <div>
                    <h2 style="margin-top:0;">Price
Trackers</h2>
                    <p style="color:var(--text-muted); margin-
bottom:0;">Automated parsing from added store URLs.</p>
                </div>
                <div class="d-flex gap-10">
                    <input type="url" id="newUrl"
placeholder="https://notino... or makeup..." style="width:
350px;">
                    <button class="btn btn-primary"
onclick="fetchOptions()">Fetch Options</button>
                    <button class="btn"
onclick="toggleManualPaste()">Manual Paste</button>
                    <button class="btn"
onclick="refreshAllPrices()">Refresh All</button>
```

```
                </div>
            </div>

            <div id="manualPastePanel" class="panel"
style="display:none; margin-top:15px; border-left: 4px solid
var(--secondary-color);">
                <h3 style="margin-top:0;">Manual HTML
Paste</h3>
                <p style="color:var(--text-muted); margin-
bottom: 10px;">If the site blocks automated access, copy the
page HTML from your browser and paste it here.</p>
                <textarea id="manualHtml" placeholder="Paste
HTML here..." style="width: 100%; height: 200px; margin-bottom:
10px; font-family: monospace;"></textarea>
                <div class="d-flex justify-end">
                    <button class="btn btn-primary"
onclick="parseManualHtml()">Parse HTML</button>
                </div>
            </div>

            <div id="variantsPanel" class="panel"
style="display:none; margin-top:15px; border-left: 4px solid
var(--primary-color);">
                <h3 style="margin-top:0;">Available
Variants</h3>
                <p style="color:var(--text-muted); margin-
bottom: 10px;">Select the formats you want to track from this
URL.</p>
                <div id="variantsContainer" class="d-flex gap-
10" style="flex-wrap: wrap;">
                    <!-- Variants will be injected here -->
```

```
                </div>
            </div>

            <div class="panel">
                <div class="d-flex justify-between align-
center" style="margin-bottom: 15px;">
                    <h3 style="margin: 0;">Price History</h3>
                </div>
                <div th:if="${perfume.prices == null or
perfume.prices.empty}" class="empty-state">
                    No prices tracked for this perfume. Paste a
product URL above to start.
                </div>

                <div th:unless="${perfume.prices == null or
perfume.prices.empty}" style="height: 300px; margin-bottom:
20px;">
                    <canvas id="priceChart"></canvas>
                </div>

                <div th:unless="${perfume.prices == null or
perfume.prices.empty}" style="overflow-x: auto;">
                    <table style="width: 100%; border-collapse:
collapse;">
                    <thead>
                        <tr>
                            <th style="padding: 10px; border-
bottom: 1px solid var(--border-color); text-align:
left;">Store</th>
```

```
                            <th style="padding: 10px; border-
bottom: 1px solid var(--border-color); text-align:
left;">Volume (ml)</th>
                            <th style="padding: 10px; border-
bottom: 1px solid var(--border-color); text-align: left;">Price
(Local)</th>
                            <th style="padding: 10px; border-
bottom: 1px solid var(--border-color); text-align: left;">Price
(USD)</th>
                            <th style="padding: 10px; border-
bottom: 1px solid var(--border-color); text-align:
left;">Price/ML (USD)</th>
                            <th style="padding: 10px; border-
bottom: 1px solid var(--border-color); text-align: left;">Last
Checked</th>
                            <th style="padding: 10px; border-
bottom: 1px solid var(--border-color); text-align:
left;">Action</th>
                        </tr>
                    </thead>
                    <tbody>
                        <tr th:each="price :
${perfume.prices}">
                            <td style="padding: 10px; border-
bottom: 1px solid var(--border-color);"
th:text="${price.storeName}">Store</td>
                            <td style="padding: 10px; border-
bottom: 1px solid var(--border-color);" th:text="${price.volume
!= null ? price.volume + ' ml' : 'N/A'}">Volume</td>
                            <td style="padding: 10px; border-
bottom: 1px solid var(--border-color); font-weight: bold;">
```

```
                                <span
th:text="${price.price}"></span> <span
th:text="${price.currency}"></span>
                                </td>
                                <td style="padding: 10px; border-
bottom: 1px solid var(--border-color);" class="price-tag"
th:text="${price.convertedPriceUsd != null ? '$' +
price.convertedPriceUsd : '-'}"></td>
                                <td style="padding: 10px; border-
bottom: 1px solid var(--border-color);"
th:text="${price.pricePerMlUsd != null ? '$' +
price.pricePerMlUsd + '/ml' : '-'}"></td>
                                <td style="padding: 10px; border-
bottom: 1px solid var(--border-color);"
th:text="${#temporals.format(price.createdAt, 'yyyy-MM-dd
HH:mm')}"></td>
                                <td style="padding: 10px; border-
bottom: 1px solid var(--border-color); display:flex; gap:
5px;">
                                    <a
th:href="${price.productUrl}" target="_blank" class="btn btn-
sm">Visit Store</a>
                                    <button class="btn btn-sm"
style="background:#dc3545; color:white;"
th:attr="onclick='deletePrice(' + ${price.id} +
')'">Delete</button>
                                </td>
                            </tr>
                        </tbody>
                    </table>
                </div>
```

```
        </div>
    </div>

    <script th:inline="javascript">
        const perfumeId = /*[[${perfume.id}]]*/ 0;

        function toggleManualPaste() {
            const panel =
document.getElementById('manualPastePanel');
            panel.style.display = panel.style.display ===
'none' ? 'block' : 'none';
        }

        function parseManualHtml() {
            const url =
document.getElementById('newUrl').value.trim();
            const html =
document.getElementById('manualHtml').value.trim();

            if(!url) {
                alert("Please enter the store URL first.");
                return;
            }
            if(!html) {
                alert("Please paste the HTML content.");
                return;
            }
```

```
document.getElementById('variantsContainer').innerHTML =
'Loading...';

document.getElementById('variantsPanel').style.display =
'block';

            fetch('/api/prices/parse-manual', {
                method: 'POST',
                headers: {
                    'Content-Type': 'application/json'
                },
                body: JSON.stringify({ url: url, html: html })
            })
            .then(res => {
                if (res.ok) return res.json();
                return res.text().then(text => { throw new
Error(text) });
            })
            .then(variants => renderVariants(variants))
            .catch(err => {
                alert("Error parsing manual HTML: " +
err.message);

document.getElementById('variantsPanel').style.display =
'none';
            });
        }

        function fetchOptions() {
            const urlInput = document.getElementById('newUrl');
```

```
            const url = urlInput.value.trim();
            if (!url) return;

            urlInput.disabled = true;

document.getElementById('variantsContainer').innerHTML =
'Loading...';

document.getElementById('variantsPanel').style.display =
'block';

            fetch('/api/prices/parse', {
                method: 'POST',
                headers: {
                    'Content-Type': 'application/json'
                },
                body: JSON.stringify({ perfumeId: perfumeId,
url: url })
            })
            .then(res => {
                if (res.ok) return res.json();
                return res.text().then(text => { throw new
Error(text) });
            })
            .then(variants => renderVariants(variants))
            .catch(err => {
                alert("Error fetching options: " +
err.message);

document.getElementById('variantsPanel').style.display =
'none';
```

```
                urlInput.disabled = false;
            });
        }

        function renderVariants(variants) {
            const container =
document.getElementById('variantsContainer');
            container.innerHTML = '';

            if(variants.length === 0) {
                container.innerHTML = 'No variants found. You
may need to check your selectors or JSON-LD.';
                document.getElementById('newUrl').disabled =
false;
                return;
            }

            variants.forEach((v, index) => {
                const card = document.createElement('div');
                card.style.border = '1px solid var(--border-
color)';
                card.style.padding = '10px';
                card.style.borderRadius = '4px';
                card.style.background = 'var(--panel-bg)';
                card.style.minWidth = '200px';

                const title = v.variantName ? v.variantName :
(v.volume ? v.volume + 'ml' : 'Unknown');

                card.innerHTML = `
```

```
                    <div style="font-weight: bold; margin-
bottom: 5px;">${title}</div>
                    <div style="color: var(--text-muted);
margin-bottom: 10px;">${v.price} ${v.currency}</div>
                    <button class="btn btn-success"
style="width: 100%; padding: 5px;"
onclick='trackVariant(${JSON.stringify(v)})'>Track
This</button>
                `;
                container.appendChild(card);
            });
            document.getElementById('newUrl').disabled = false;
        }

        function trackVariant(variantData) {
            fetch('/api/prices/track', {
                method: 'POST',
                headers: {
                    'Content-Type': 'application/json'
                },
                body: JSON.stringify(variantData)
            })
            .then(res => {
                if(res.ok) window.location.reload();
                else return res.text().then(text => { throw new
Error(text) });
            })
            .catch(err => alert("Error saving tracker: " +
err.message));
        }
```

```
        function deletePrice(priceId) {
            if(!confirm("Are you sure you want to delete this
tracked price?")) return;

            fetch('/api/prices/' + priceId, {
                method: 'DELETE'
            })
            .then(res => {
                if(res.ok) window.location.reload();
                else return res.text().then(text => { throw new
Error(text) });
            })
            .catch(err => alert("Error deleting price: " +
err.message));
        }

        function refreshAllPrices() {
            alert('Refreshes happen automatically in
background. Manual trigger can be implemented here.');
        }

        // Chart.js Visualization Logic
        document.addEventListener('DOMContentLoaded',
function() {
            const ctx = document.getElementById('priceChart');
            if (!ctx) return;

            const datasets = [];
            const allDates = new Set();

            /*[# th:each="price : ${perfume.prices}"]*/
```

```
                var dataPoints = [];

                /*[# th:each="hist : ${price.history}"]*/
                    dataPoints.push({
                        x:
/*[[${#temporals.format(hist.timestamp, 'yyyy-MM-dd
HH:mm')}]]*/ '',
                        y: /*[[${hist.price}]]*/ 0
                    });

allDates.add(/*[[${#temporals.format(hist.timestamp, 'yyyy-MM-
dd HH:mm')}]]*/ '');
                /*[/]*/

                if(dataPoints.length === 0) {
                    var createDate =
/*[[${#temporals.format(price.createdAt, 'yyyy-MM-dd
HH:mm')}]]*/ '';
                    dataPoints.push({ x: createDate, y:
/*[[${price.price}]]*/ 0 });
                    allDates.add(createDate);
                }

                // Add current price as the latest point
                var currentDate =
/*[[${#temporals.format(#temporals.createNow(), 'yyyy-MM-dd
HH:mm')}]]*/ '';
                dataPoints.push({ x: currentDate, y:
/*[[${price.price}]]*/ 0 });
                allDates.add(currentDate);
```

```
                // generate distinct colors based on store and
variant name
                var storeName = /*[[${price.storeName}]]*/ '';
                var variantName = /*[[${price.variantName}]]*/
'';
                var volumeVal = /*[[${price.volume != null ?
price.volume : ''}]]*/ '';

                var str = storeName + '-' + variantName;
                var hash = 0;
                for (var i = 0; i < str.length; i++) hash =
str.charCodeAt(i) + ((hash << 5) - hash);
                var color = '#' + (hash &
0x00FFFFFF).toString(16).padStart(6, '0');

                var displayLabel = storeName;
                if (variantName && variantName !== 'null' &&
variantName !== '' && variantName !== volumeVal + ' ml' &&
variantName !== volumeVal + 'ml') {
                    displayLabel += ' - ' + variantName;
                }
                if (volumeVal && volumeVal !== 'null') {
                    displayLabel += ' ' + volumeVal + 'ml';
                }

                datasets.push({
                    label: displayLabel,
                    data: dataPoints,
                    fill: false,
                    tension: 0.1,
                    borderColor: color,
```

```
            backgroundColor: color,
            borderWidth: 2,
            stepped: true
        });
    /*[/]*/

    const sortedLabels = Array.from(allDates).sort();

    datasets.forEach(ds => {
        const map = {};
        ds.data.forEach(dp => map[dp.x] = dp.y);

        let lastKnown = null;
        ds.data = sortedLabels.map(l => {
            if (map[l] !== undefined) {
                lastKnown = map[l];
            }
            return lastKnown;
        });
    });

    if (datasets.length > 0) {
        new Chart(ctx, {
            type: 'line',
            data: {
                labels: sortedLabels,
                datasets: datasets
            },
            options: {
                responsive: true,
                maintainAspectRatio: false,
```

```
                        spanGaps: true,
                        scales: {
                            y: { beginAtZero: false }
                        }
                    }
                });
            }
        });
    </script>
</body>
</html>
```

Perfumes.html

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{layout :: head('Perfumes - PPA')}"></head>
<body>
    <div th:replace="~{layout :: sidebar}"></div>

    <div class="main-content">
        <div class="topbar">
            <h1 class="page-title">Perfumes Catalog</h1>
            <div style="flex:1"></div>
            <a th:href="@{/add-perfume}" class="btn btn-
primary">New Perfume</a>
        </div>

        <div class="content-area">
            <div class="panel">
```

```
                <div th:if="${perfumes.empty}" class="empty-
state">
                    No perfumes tracked yet. Add one to start.
                </div>

                <table th:unless="${perfumes.empty}">
                    <thead>
                        <tr>
                            <th>ID</th>
                            <th>Brand</th>
                            <th>Name</th>
                            <th>Gender</th>
                            <th>Added</th>
                            <th>Action</th>
                        </tr>
                    </thead>
                    <tbody>
                        <tr th:each="p : ${perfumes}">
                            <td th:text="${p.id}"></td>
                            <td th:text="${p.brand}"></td>
                            <td th:text="${p.name}"></td>
                            <td th:text="${p.gender}"></td>
                            <td
th:text="${#temporals.format(p.createdAt, 'yyyy-MM-dd')}"></td>
                            <td>
                                <a
th:href="@{/perfumes/{id}(id=${p.id})}" class="btn btn-sm">View
Details</a>
                            </td>
                        </tr>
                    </tbody>
```

```
                </table>
            </div>
        </div>
    </div>
</body>
</html>
```

Store.html

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{layout :: head('Stores - PPA')}"></head>
<body>
    <div th:replace="~{layout :: sidebar}"></div>

    <div class="main-content">
        <div class="topbar">
            <h1 class="page-title">Registered Stores</h1>
        </div>

        <div class="content-area">

            <div class="panel d-flex justify-between align-
center">
                <div>
                    <h2 style="margin-top:0;">Supported
Parsers</h2>
                    <p style="color:var(--text-muted); margin-
bottom:0;">Stores discovered automatically during tracking.</p>
                </div>
```

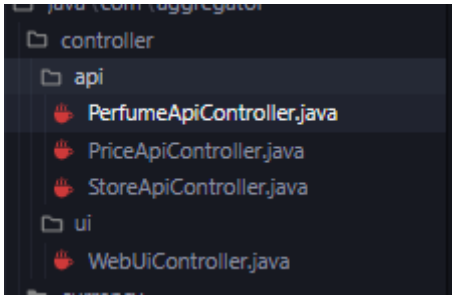
```
            </div>

            <div class="panel">
                <div th:if="${stores.empty}" class="empty-
state">
                    No stores tracked yet.
                </div>

                <table th:unless="${stores.empty}">
                    <thead>
                        <tr>
                            <th>ID</th>
                            <th>Store Name</th>
                            <th>Parser Implementation</th>
                        </tr>
                    </thead>
                    <tbody>
                        <tr th:each="s : ${stores}">
                            <td th:text="${s.id}"></td>
                            <td th:text="${s.name}"></td>
                            <td th:text="${s.parserType}"></td>
                        </tr>
                    </tbody>
                </table>
            </div>
        </div>
    </div>
</body>
</html>
```

Application.yaml

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head th:replace="~{layout :: head('Stores - PPA')}"></head>
<body>
    <div th:replace="~{layout :: sidebar}"></div>

    <div class="main-content">
        <div class="topbar">
            <h1 class="page-title">Registered Stores</h1>
        </div>

        <div class="content-area">

            <div class="panel d-flex justify-between align-
center">
                <div>
                    <h2 style="margin-top:0;">Supported
Parsers</h2>
                    <p style="color:var(--text-muted); margin-
bottom:0;">Stores discovered automatically during tracking.</p>
                </div>
            </div>

            <div class="panel">
                <div th:if="${stores.empty}" class="empty-
state">
                    No stores tracked yet.
                </div>

                <table th:unless="${stores.empty}">
                    <thead>
```

```
                        <tr>
                            <th>ID</th>
                            <th>Store Name</th>
                            <th>Parser Implementation</th>
                        </tr>
                    </thead>
                    <tbody>
                        <tr th:each="s : ${stores}">
                            <td th:text="${s.id}"></td>
                            <td th:text="${s.name}"></td>
                            <td th:text="${s.parserType}"></td>
                        </tr>
                    </tbody>
                </table>
            </div>
        </div>
    </div>
</body>
</html>
```

/controllers - файли для виконання рест запитів та керуванням ui



Файл для виконання операцій з парфумами

```java
package com.aggregator.controller.api;

import com.aggregator.dto.PerfumeDto;
import com.aggregator.service.PerfumeService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/perfumes")
public class PerfumeApiController {

    private final PerfumeService perfumeService;

    @Autowired
    public PerfumeApiController(PerfumeService perfumeService)
{
        this.perfumeService = perfumeService;
    }

    @GetMapping
    public ResponseEntity<List<PerfumeDto>> getAllPerfumes() {
        return
ResponseEntity.ok(perfumeService.getAllPerfumes());
    }

    @GetMapping("/{id}")
    public ResponseEntity<PerfumeDto>
getPerfumeById(@PathVariable Long id) {
        PerfumeDto dto = perfumeService.getPerfumeDetails(id);
```

```
        if (dto != null) {
            return ResponseEntity.ok(dto);
        }
        return ResponseEntity.notFound().build();
    }

    @PostMapping
    public ResponseEntity<PerfumeDto>
createPerfume(@RequestBody PerfumeDto dto) {
        return
ResponseEntity.ok(perfumeService.createPerfume(dto));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deletePerfume(@PathVariable
Long id) {
        perfumeService.deletePerfume(id);
        return ResponseEntity.ok().build();
    }
```

Файл для виконання операцій парсингу варіантів, отриманням ціни, відслідковуванням змін

```
package com.aggregator.controller.api;

import com.aggregator.dto.PerfumePriceDto;
import com.aggregator.service.PriceAggregationService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import lombok.extern.slf4j.Slf4j;
```

```java
import java.util.List;
import java.util.Map;

@Slf4j
@RestController
@RequestMapping("/api/prices")
public class PriceApiController {

    private final PriceAggregationService aggregationService;
    private final com.aggregator.service.ManualHtmlService
manualHtmlService;

    @Autowired
    public PriceApiController(PriceAggregationService
aggregationService, com.aggregator.service.ManualHtmlService
manualHtmlService) {
        this.aggregationService = aggregationService;
        this.manualHtmlService = manualHtmlService;
    }

    @PostMapping("/parse")
    public ResponseEntity<?> fetchVariants(@RequestBody
Map<String, Object> payload) {
        try {
            Long perfumeId =
Long.valueOf(payload.get("perfumeId").toString());
            String url = (String) payload.get("url");

            List<PerfumePriceDto> variants =
aggregationService.fetchVariants(perfumeId, url);
            return ResponseEntity.ok(variants);
```

```
        } catch (IllegalArgumentException e) {
            return
ResponseEntity.badRequest().body(e.getMessage());
        } catch (Exception e) {
            log.error("Error fetching variants: ", e);
            return
ResponseEntity.internalServerError().body("An unexpected error
occurred while parsing the URL.");
        }
    }

    @PostMapping("/parse-manual")
    public ResponseEntity<?> parseManualHtml(@RequestBody
Map<String, Object> payload) {
        try {
            String url = (String) payload.get("url");
            String html = (String) payload.get("html");

            List<PerfumePriceDto> variants =
manualHtmlService.parseManualHtml(url, html);
            return ResponseEntity.ok(variants);
        } catch (Exception e) {
            log.error("Error parsing manual HTML: ", e);
            return
ResponseEntity.internalServerError().body("An unexpected error
occurred while parsing the provided HTML.");
        }
    }

    @PostMapping("/track")
```

```
    public ResponseEntity<?> trackVariant(@RequestBody
PerfumePriceDto variantData) {
        try {
            PerfumePriceDto saved =
aggregationService.trackVariant(variantData);
            return ResponseEntity.ok(saved);
        } catch (IllegalArgumentException e) {
            return
ResponseEntity.badRequest().body(e.getMessage());
        } catch (Exception e) {
            log.error("Error tracking variant: ", e);
            return
ResponseEntity.internalServerError().body("An unexpected error
occurred while tracking the variant.");
        }
    }
    @DeleteMapping("/{id}")
    public ResponseEntity<?> deletePrice(@PathVariable Long id)
{
        try {
            aggregationService.deletePrice(id);
            return ResponseEntity.ok().build();
        } catch (Exception e) {
            log.error("Error deleting price: ", e);
            return
ResponseEntity.internalServerError().body("An unexpected error
occurred while deleting the price.");
        }
    }
}
```

Файл для керуванням магазинів

```java
package com.aggregator.controller.api;

import com.aggregator.dto.StoreDto;
import com.aggregator.service.StoreService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/stores")
public class StoreApiController {

    private final StoreService storeService;

    @Autowired
    public StoreApiController(StoreService storeService) {
        this.storeService = storeService;
    }

    @GetMapping
    public ResponseEntity<List<StoreDto>> getAllStores() {
        return ResponseEntity.ok(storeService.getAllStores());
    }

    @PostMapping
```

```
    public ResponseEntity<StoreDto> createStore(@RequestBody
StoreDto dto) {
        return
ResponseEntity.ok(storeService.createStore(dto));
    }
}
```

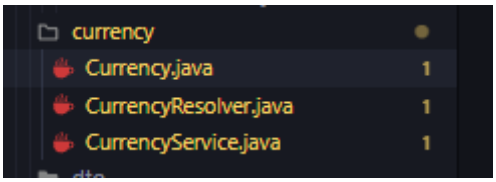
Файл для навігацію по застосунку

```
package com.aggregator.controller.ui;

import com.aggregator.service.PerfumeService;
import com.aggregator.service.StoreService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;

@Controller
public class WebUiController {

    private final PerfumeService perfumeService;
    private final StoreService storeService;

    @Autowired
    public WebUiController(PerfumeService perfumeService,
StoreService storeService) {
```

```
        this.perfumeService = perfumeService;
        this.storeService = storeService;
    }

    @GetMapping({"/", "/perfumes"})
    public String perfumes(Model model) {
        model.addAttribute("perfumes",
perfumeService.getAllPerfumes());
        return "perfumes";
    }

    @GetMapping("/perfumes/{id}")
    public String perfumeDetails(@PathVariable Long id, Model
model) {
        model.addAttribute("perfume",
perfumeService.getPerfumeDetails(id));
        model.addAttribute("stores",
storeService.getAllStores());
        return "perfume-details";
    }

    @GetMapping("/add-perfume")
    public String addPerfume(Model model) {
        return "add-perfume";
    }

    @GetMapping("/stores")
    public String stores(Model model) {
        model.addAttribute("stores",
storeService.getAllStores());
        return "stores";
```

```
    }
}
```

/currency - пакет створенний для оперуванням різною валютою



Енумерація можливих варіантів валюти з магазинів в яких я зазвичай дивлюсь парфуми

```
package com.aggregator.currency;

public enum Currency {
    USD,
    EUR,
    UAH,
    PLN,
    UNKNOWN
}
```

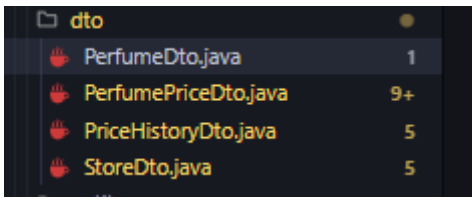
Резолвер який надає можливість з тексту отримати варіант енумерації

```
package com.aggregator.currency;

public class CurrencyResolver {

    public static Currency resolve(String text) {
        if (text == null) {
```

```
            return Currency.UNKNOWN;
        }

        String lowerText = text.toLowerCase();

        if (lowerText.contains("$") ||
lowerText.contains("usd")) {
            return Currency.USD;
        } else if (lowerText.contains("€") ||
lowerText.contains("eur")) {
            return Currency.EUR;
        } else if (lowerText.contains("₴") ||
lowerText.contains("грн") || lowerText.contains("uah")) {
            return Currency.UAH;
        } else if (lowerText.contains("zł") ||
lowerText.contains("pln")) {
            return Currency.PLN;
        }

        return Currency.UNKNOWN;
    }
}
```

Тимчасовий сервіс, який дозволяє конвертувати ціни з долларів та до долларів. В ідеалі згодом замінити на виклик API, який дозволяэ конвертувати валюти

/dto - файл де зібрані всі dto об’єкти




```java
package com.aggregator.dto;

import lombok.Data;
import java.time.LocalDateTime;
import java.util.List;

@Data
public class PerfumeDto {
    private Long id;
    private String name;
    private String brand;
    private String gender;
    private LocalDateTime createdAt;
    private List<PerfumePriceDto> prices;
}
```

```java
package com.aggregator.dto;

import com.aggregator.currency.Currency;
import lombok.Data;
import java.math.BigDecimal;
import java.time.LocalDateTime;

@Data
public class PerfumePriceDto {
    private Long id;
```

```
    private BigDecimal price;
    private Currency currency;
    private Integer volume;
    private String variantName;
    private String productUrl;
    private LocalDateTime createdAt;

    private Long perfumeId;
    private String perfumeName;

    private Long storeId;
    private String storeName;

    private BigDecimal convertedPriceUsd;
    private BigDecimal pricePerMlUsd;

    private java.util.List<PriceHistoryDto> history = new
java.util.ArrayList<>();
}

package com.aggregator.dto;

import com.aggregator.currency.Currency;
import lombok.Data;
import java.math.BigDecimal;
import java.time.LocalDateTime;

@Data
public class PriceHistoryDto {
    private Long id;
    private BigDecimal price;
    private Currency currency;
```

```
    private LocalDateTime timestamp;
}
```

```
package com.aggregator.dto;

import lombok.Data;

@Data
public class StoreDto {
    private Long id;
    private String name;
    private String website;
    private String parserType;
}
```

/entity - репрезентація сутностей з бази данних

```
package com.aggregator.entity;

import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

@Entity
@Getter
@Setter
```

```
public class Perfume {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String brand;
    private String gender;

    @Column(name = "created_at")
    private LocalDateTime createdAt;

    @OneToMany(mappedBy = "perfume", cascade = CascadeType.ALL,
orphanRemoval = true)
    private List<PerfumePrice> prices = new ArrayList<>();

    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();
    }
}

package com.aggregator.entity;

import com.aggregator.currency.Currency;
import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;
import java.math.BigDecimal;
import java.time.LocalDateTime;
import java.util.ArrayList;
```

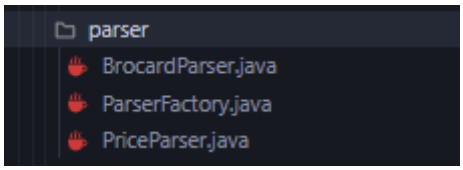
```java
import java.util.List;

@Entity
@Table(name = "perfume_price")
@Getter
@Setter
public class PerfumePrice {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private BigDecimal price;

    @Enumerated(EnumType.STRING)
    private Currency currency;

    private Integer volume; // in ml

    @Column(name = "variant_name")
    private String variantName;

    @Column(name = "product_url", length = 2048)
    private String productUrl;

    @Column(name = "created_at")
    private LocalDateTime createdAt;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "perfume_id")
    private Perfume perfume;

    @ManyToOne(fetch = FetchType.LAZY)
```

```
    @JoinColumn(name = "store_id")
    private Store store;

    @OneToMany(mappedBy = "perfumePrice", cascade =
CascadeType.ALL, orphanRemoval = true)
    private List<PriceHistory> history = new ArrayList<>();

    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();
    }
}

package com.aggregator.entity;

import com.aggregator.currency.Currency;
import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;
import java.math.BigDecimal;
import java.time.LocalDateTime;

@Entity
@Table(name = "price_history")
@Getter
@Setter
public class PriceHistory {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```
    private BigDecimal price;

    @Enumerated(EnumType.STRING)
    private Currency currency;

    private LocalDateTime timestamp;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "perfume_price_id")
    private PerfumePrice perfumePrice;
}

package com.aggregator.entity;

import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;
import java.util.ArrayList;
import java.util.List;

@Entity
@Getter
@Setter
public class Store {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String website;

    @Column(name = "parser_type")
```

```
    private String parserType;

    @OneToMany(mappedBy = "store", cascade = CascadeType.ALL,
orphanRemoval = true)
    private List<PerfumePrice> prices = new ArrayList<>();
}
```

/parser - файл з фабрикойю та стратегіями для парсингу сторінок, так як різні магазини мають різну структуру і різні можливості обробляти це я вирішив за допомогою такого патерну як фабрика, що дозволить в майбутньому з легкістю просто додавати стратегії парсингу під різні магазини




```
package com.aggregator.parser;

import com.aggregator.currency.Currency;
import com.aggregator.currency.CurrencyResolver;
import com.aggregator.dto.PerfumePriceDto;
import com.aggregator.util.PriceExtractor;
import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import lombok.extern.slf4j.Slf4j;
import org.jsoup.Jsoup;
import org.jsoup.nodes.Document;
import org.jsoup.nodes.Element;
import org.jsoup.select.Elements;
import org.springframework.stereotype.Component;

import java.util.ArrayList;
```

```
import java.util.List;

@Slf4j
@Component
public class BrocardParser implements PriceParser {

    private final ObjectMapper objectMapper = new
ObjectMapper();

    @Override
    public String getStoreName() {
        return "Brocard";
    }

    @Override
    public boolean supports(String url) {
        return url != null && url.contains("brocard.ua");
    }

    @Override
    public List<PerfumePriceDto> parseVariants(String
productUrl) {
        try {
            Document doc = Jsoup.connect(productUrl)
                    .userAgent("Mozilla/5.0 (Windows NT 10.0;
Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/122.0.0.0 Safari/537.36")
                    .header("Accept-Language", "uk,en-
US;q=0.9,en;q=0.8")
                    .timeout(15000)
                    .ignoreHttpErrors(true)
```

```
                    .get();

            return parseDocument(doc, productUrl);
        } catch (Exception e) {
            log.error("Error connecting to Brocard URL {}: {}",
productUrl, e.getMessage());
            return null;
        }
    }

    @Override
    public List<PerfumePriceDto> parseDocument(Document doc,
String productUrl) {
        List<PerfumePriceDto> variants = new ArrayList<>();
        String mainName = "";
        Element nameEl = doc.selectFirst("h1.product-title, h1,
.product-item__name");
        if (nameEl != null) {
            mainName = nameEl.text().trim();
        } else {
            mainName = doc.title().replace(" - BROCARD",
"").trim();
        }

        // 1. Try JSON-LD
        try {
            Elements scripts =
doc.select("script[type=application/ld+json]");
            for (Element script : scripts) {
                String html = script.html();
                if (html.contains("\"Product\"")) {
```

```
                    JsonNode node =
objectMapper.readTree(html);
                    if (node.isArray()) {
                        for (JsonNode item : node) {
                            extractFromJsonLd(item, variants,
mainName, productUrl);
                        }
                    } else {
                        extractFromJsonLd(node, variants,
mainName, productUrl);
                    }
                }
            }
        } catch (Exception e) {
            log.warn("Error parsing Brocard JSON-LD: {}",
e.getMessage());
        }

        if (!variants.isEmpty()) return variants;

        // 2. DOM Fallback
        Element priceEl = doc.selectFirst(".price-format
.price, .product-price__big, .price .wysiwyg, .product-
item__price");
        if (priceEl != null) {
            PerfumePriceDto dto = new PerfumePriceDto();
            dto.setProductUrl(productUrl);
            dto.setStoreName(getStoreName());
            dto.setPerfumeName(mainName);

            String priceRaw = priceEl.text();
```

```
            dto.setPrice(PriceExtractor.extract(priceRaw));

            String currencyText = priceRaw;
            Element currencyEl = doc.selectFirst(".currency-
symbol, .price-currency");
            if (currencyEl != null) currencyText =
currencyEl.text();

dto.setCurrency(CurrencyResolver.resolve(currencyText));

            // Get the ACTIVE Selected Volume
            Element activeVolumeEl = doc.selectFirst(".swatch-
wrapper .swatch-label, .options-holder .selected .swatch-label,
.swatch-attribute-selected-option");
            if (activeVolumeEl != null) {
                String volText = activeVolumeEl.text().trim();
                dto.setVariantName(volText);

dto.setVolume(PriceExtractor.extractVolume(volText));
            } else {
                dto.setVariantName("Standard");

dto.setVolume(PriceExtractor.extractVolume(mainName));
            }

            if (dto.getPrice() != null) {
                variants.add(dto);
            }
        }

        return variants;
```

```
    }

    private void extractFromJsonLd(JsonNode node,
List<PerfumePriceDto> variants, String mainName, String
productUrl) {
        if (node.has("@type") &&
node.get("@type").asText().equals("Product") &&
node.has("offers")) {
            JsonNode offersNode = node.get("offers");

            if (offersNode.has("offers") &&
offersNode.get("offers").isArray()) {
                for (JsonNode offer : offersNode.get("offers"))
{
                    addVariantFromOffer(offer, variants,
mainName, productUrl);
                }
            } else if (offersNode.isArray()) {
                for (JsonNode offer : offersNode) {
                    addVariantFromOffer(offer, variants,
mainName, productUrl);
                }
            } else {
                addVariantFromOffer(offersNode, variants,
mainName, productUrl);
            }
        }
    }
```

```
    private void addVariantFromOffer(JsonNode offer,
List<PerfumePriceDto> variants, String mainName, String
productUrl) {
        try {
            if (offer.has("price") &&
!offer.get("price").isNull()) {
                PerfumePriceDto dto = new PerfumePriceDto();
                dto.setProductUrl(productUrl);
                dto.setStoreName(getStoreName());
                dto.setPerfumeName(mainName);


dto.setPrice(PriceExtractor.extract(offer.get("price").asText()
));
                if (offer.has("priceCurrency")) {

dto.setCurrency(CurrencyResolver.resolve(offer.get("priceCurren
cy").asText()));
                } else {
                    dto.setCurrency(Currency.UAH);
                }

                String vName = offer.has("name") ?
offer.get("name").asText() : "Standard";
                dto.setVariantName(vName);

dto.setVolume(PriceExtractor.extractVolume(vName));

                if (dto.getVolume() == null) {

dto.setVolume(PriceExtractor.extractVolume(mainName));
```

```
                }

                if (dto.getPrice() != null) {
                    variants.add(dto);
                }
            }
        } catch (Exception e) {}
    }
}

package com.aggregator.parser;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

import java.util.List;
import java.util.Optional;

@Component
public class ParserFactory {

    private final List<PriceParser> parsers;

    @Autowired
    public ParserFactory(List<PriceParser> parsers) {
        this.parsers = parsers;
    }

    public Optional<PriceParser> getParser(String url) {
        return parsers.stream()
                .filter(p -> p.supports(url))
                .findFirst();
```

```
    }
}

package com.aggregator.parser;

import com.aggregator.dto.PerfumePriceDto;
import org.jsoup.nodes.Document;
import java.util.List;

public interface PriceParser {
    String getStoreName();
    boolean supports(String url);
    List<PerfumePriceDto> parseVariants(String productUrl);
    List<PerfumePriceDto> parseDocument(Document doc, String
productUrl);
}
```

/repository - репозиторії проєкту для взаємодії з БД

```
package com.aggregator.repository;

import com.aggregator.entity.PerfumePrice;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.Optional;

@Repository
public interface PerfumePriceRepository extends
JpaRepository<PerfumePrice, Long> {
```

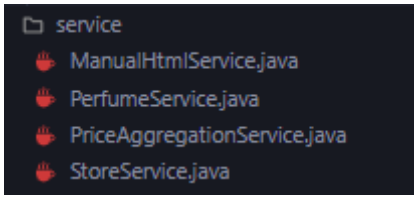
```
    List<PerfumePrice> findByPerfumeId(Long perfumeId);
    Optional<PerfumePrice>
findByPerfumeIdAndStoreIdAndVolume(Long perfumeId, Long
storeId, Integer volume);
}

package com.aggregator.repository;

import com.aggregator.entity.Perfume;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface PerfumeRepository extends
JpaRepository<Perfume, Long> {
}
```

```
package com.aggregator.repository;

import com.aggregator.entity.PriceHistory;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;

@Repository
public interface PriceHistoryRepository extends
JpaRepository<PriceHistory, Long> {
    List<PriceHistory>
findByPerfumePriceIdOrderByTimestampDesc(Long perfumePriceId);
```

```
}
```

```
package com.aggregator.repository;

import com.aggregator.entity.Store;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.Optional;

@Repository
public interface StoreRepository extends JpaRepository<Store,
Long> {
    Optional<Store> findByName(String name);
}
```

/service - файли з сервісами, які використовуються в проєкті




```
package com.aggregator.service;

import com.aggregator.dto.PerfumePriceDto;
import com.aggregator.parser.ParserFactory;
import com.aggregator.parser.PriceParser;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
```

```java
import org.jsoup.Jsoup;
import org.jsoup.nodes.Document;
import org.springframework.stereotype.Service;

import java.util.Collections;
import java.util.List;

@Slf4j
@Service
@RequiredArgsConstructor
public class ManualHtmlService {

    private final ParserFactory parserFactory;

    /**
     * Parses product variants from provided HTML content and URL.
     *
     * @param url The product URL to identify the correct parser.
     * @param html The raw HTML content pasted by the user.
     * @return List of parsed product variants.
     */
    public List<PerfumePriceDto> parseManualHtml(String url, String html) {
        if (url == null || html == null || html.trim().isEmpty()) {
            return Collections.emptyList();
        }

        try {
```

```
            PriceParser parser = parserFactory.getParser(url)
                    .orElseThrow(() -> new
IllegalArgumentException("Unsupported store URL: " + url));
            Document doc = Jsoup.parse(html, url);
            return parser.parseDocument(doc, url);
        } catch (Exception e) {
            log.error("Error parsing manual HTML for URL {}:
{}", url, e.getMessage());
            return Collections.emptyList();
        }
    }
}

package com.aggregator.service;

import com.aggregator.currency.CurrencyService;
import com.aggregator.dto.PerfumeDto;
import com.aggregator.dto.PerfumePriceDto;
import com.aggregator.dto.PriceHistoryDto;
import com.aggregator.entity.Perfume;
import com.aggregator.entity.PerfumePrice;
import com.aggregator.repository.PerfumeRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import
org.springframework.transaction.annotation.Transactional;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.List;
import java.util.stream.Collectors;
```

```java
@Service
public class PerfumeService {

    private final PerfumeRepository perfumeRepository;
    private final CurrencyService currencyService;

    @Autowired
    public PerfumeService(PerfumeRepository perfumeRepository,
CurrencyService currencyService) {
        this.perfumeRepository = perfumeRepository;
        this.currencyService = currencyService;
    }

    @Transactional(readOnly = true)
    public List<PerfumeDto> getAllPerfumes() {
        return perfumeRepository.findAll().stream()
                .map(p -> mapToDto(p, false))
                .collect(Collectors.toList());
    }

    @Transactional(readOnly = true)
    public PerfumeDto getPerfumeDetails(Long id) {
        return perfumeRepository.findById(id)
                .map(p -> mapToDto(p, true))
                .orElse(null);
    }

    @Transactional
    public PerfumeDto createPerfume(PerfumeDto dto) {
        Perfume perfume = new Perfume();
```

```
        perfume.setName(dto.getName());
        perfume.setBrand(dto.getBrand());
        perfume.setGender(dto.getGender());

        Perfume saved = perfumeRepository.save(perfume);
        return mapToDto(saved, false);
    }

    @Transactional
    public void deletePerfume(Long id) {
        perfumeRepository.deleteById(id);
    }

    private PerfumeDto mapToDto(Perfume perfume, boolean
includePrices) {
        PerfumeDto dto = new PerfumeDto();
        dto.setId(perfume.getId());
        dto.setName(perfume.getName());
        dto.setBrand(perfume.getBrand());
        dto.setGender(perfume.getGender());
        dto.setCreatedAt(perfume.getCreatedAt());

        if (includePrices && perfume.getPrices() != null) {
            List<PerfumePriceDto> priceDtos =
perfume.getPrices().stream()
                    .map(this::mapPriceToDto)
                    .collect(Collectors.toList());
            dto.setPrices(priceDtos);
        }

        return dto;
```

```
    }

    private PerfumePriceDto mapPriceToDto(PerfumePrice price) {
        PerfumePriceDto dto = new PerfumePriceDto();
        dto.setId(price.getId());
        dto.setPrice(price.getPrice());
        dto.setCurrency(price.getCurrency());
        dto.setVolume(price.getVolume());
        dto.setProductUrl(price.getProductUrl());
        dto.setCreatedAt(price.getCreatedAt());

        dto.setPerfumeId(price.getPerfume().getId());
        dto.setPerfumeName(price.getPerfume().getName());

        dto.setStoreId(price.getStore().getId());
        dto.setStoreName(price.getStore().getName());

        // Calculate dynamic values
        if (price.getPrice() != null && price.getCurrency() !=
null) {
            BigDecimal usdPrice =
currencyService.convertToUsd(price.getPrice(),
price.getCurrency());
            dto.setConvertedPriceUsd(usdPrice);

            if (usdPrice != null && price.getVolume() != null
&& price.getVolume() > 0) {
                BigDecimal mlPrice = usdPrice.divide(new
BigDecimal(price.getVolume()), 4, RoundingMode.HALF_UP);
                dto.setPricePerMlUsd(mlPrice);
            }
```

```
        }

        if (price.getHistory() != null) {
            java.util.List<PriceHistoryDto> hList =
price.getHistory().stream().map(h -> {
                PriceHistoryDto hd = new PriceHistoryDto();
                hd.setId(h.getId());
                hd.setPrice(h.getPrice());
                hd.setCurrency(h.getCurrency());
                hd.setTimestamp(h.getTimestamp());
                return hd;
            }).collect(Collectors.toList());
            dto.setHistory(hList);
        }

        return dto;
    }
}

package com.aggregator.service;

import com.aggregator.dto.PerfumePriceDto;
import com.aggregator.entity.Perfume;
import com.aggregator.entity.PerfumePrice;
import com.aggregator.entity.PriceHistory;
import com.aggregator.entity.Store;
import com.aggregator.parser.ParserFactory;
import com.aggregator.parser.PriceParser;
import com.aggregator.repository.PerfumePriceRepository;
import com.aggregator.repository.PerfumeRepository;
import com.aggregator.repository.StoreRepository;
```

```java
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Service;
import
org.springframework.transaction.annotation.Transactional;

import java.time.LocalDateTime;
import java.util.List;
import java.util.Optional;

@Slf4j
@Service
public class PriceAggregationService {

    private final ParserFactory parserFactory;
    private final PerfumeRepository perfumeRepository;
    private final StoreRepository storeRepository;
    private final PerfumePriceRepository
perfumePriceRepository;

    @Autowired
    public PriceAggregationService(ParserFactory parserFactory,
PerfumeRepository perfumeRepository, StoreRepository
storeRepository, PerfumePriceRepository perfumePriceRepository)
{
        this.parserFactory = parserFactory;
        this.perfumeRepository = perfumeRepository;
        this.storeRepository = storeRepository;
        this.perfumePriceRepository = perfumePriceRepository;
    }
```

```
    @Transactional
    public List<PerfumePriceDto> fetchVariants(Long perfumeId,
String productUrl) {
        Optional<PriceParser> parserOpt =
parserFactory.getParser(productUrl);
        if (parserOpt.isEmpty()) {
            throw new IllegalArgumentException("No suitable
parser found for URL: " + productUrl);
        }

        // This just scrapes the page and returns combinations,
doesn't save to DB yet
        List<PerfumePriceDto> variants =
parserOpt.get().parseVariants(productUrl);
        if (variants == null || variants.isEmpty()) {
            throw new IllegalArgumentException("Could not
extract any variant options from this URL.");
        }

        variants.forEach(v -> v.setPerfumeId(perfumeId));
        return variants;
    }

    @Transactional
    public PerfumePriceDto trackVariant(PerfumePriceDto
parsedData) {
        Perfume perfume =
perfumeRepository.findById(parsedData.getPerfumeId())
                .orElseThrow(() -> new
IllegalArgumentException("Perfume not found"));
```

```
        if (parsedData.getPrice() == null) {
            throw new IllegalArgumentException("Cannot track
variant without a price.");
        }

        // Find or create store
        Store store =
storeRepository.findByName(parsedData.getStoreName())
                .orElseGet(() -> {
                    Store newStore = new Store();

newStore.setName(parsedData.getStoreName());

newStore.setParserType(parsedData.getStoreName() + "Parser");
// Simple guess based on store name
                    return storeRepository.save(newStore);
                });

        // Try to find exact variant tracked
        Optional<PerfumePrice> existingPriceOpt =
perfumePriceRepository.findAll().stream()
                .filter(p ->
p.getPerfume().getId().equals(perfume.getId())
                        &&
p.getStore().getId().equals(store.getId())
                        && (p.getVariantName() != null &&
p.getVariantName().equals(parsedData.getVariantName())))
                .findFirst();

        PerfumePrice perfumePrice;
```

```
        if (existingPriceOpt.isPresent()) {
            perfumePrice = existingPriceOpt.get();
            // Record history if price changed
            if (perfumePrice.getPrice() != null &&
perfumePrice.getPrice().compareTo(parsedData.getPrice()) != 0)
{
                PriceHistory history = new PriceHistory();
                history.setPrice(perfumePrice.getPrice());

history.setCurrency(perfumePrice.getCurrency());
                history.setTimestamp(LocalDateTime.now());
                history.setPerfumePrice(perfumePrice);
                perfumePrice.getHistory().add(history);
            }
            // Update to new values
            perfumePrice.setPrice(parsedData.getPrice());
            perfumePrice.setCurrency(parsedData.getCurrency());
            perfumePrice.setVolume(parsedData.getVolume());
        } else {
            perfumePrice = new PerfumePrice();
            perfumePrice.setPerfume(perfume);
            perfumePrice.setStore(store);
            perfumePrice.setVolume(parsedData.getVolume());

perfumePrice.setVariantName(parsedData.getVariantName());

perfumePrice.setProductUrl(parsedData.getProductUrl());
            perfumePrice.setPrice(parsedData.getPrice());
            perfumePrice.setCurrency(parsedData.getCurrency());

            // Record initial history
```

```
            PriceHistory history = new PriceHistory();
            history.setPrice(parsedData.getPrice());
            history.setCurrency(parsedData.getCurrency());
            history.setTimestamp(LocalDateTime.now());
            history.setPerfumePrice(perfumePrice);
            perfumePrice.getHistory().add(history);
        }

        PerfumePrice saved =
perfumePriceRepository.save(perfumePrice);

        parsedData.setId(saved.getId());
        return parsedData;
    }

    // Run every 6 hours
    @Scheduled(fixedRate = 21600000)
    @Transactional
    public void updateAllPrices() {
        log.info("Starting scheduled price update for all
products...");
        List<PerfumePrice> allPrices =
perfumePriceRepository.findAll();

        int successCount = 0;
        int errorCount = 0;

        for (PerfumePrice p : allPrices) {
            if ("Manual".equals(p.getStore().getParserType()))
continue; // Skip manual entries
```

```
            try {
                // To fetch an updated price for this tracked
variant, we refetch all from the URL
                // and match the same variantName
                Optional<PriceParser> parserOpt =
parserFactory.getParser(p.getProductUrl());
                if (parserOpt.isPresent()) {
                    List<PerfumePriceDto> latestOpts =
parserOpt.get().parseVariants(p.getProductUrl());

                    Optional<PerfumePriceDto> match =
latestOpts.stream()
                        .filter(v -> v.getVariantName() != null
&& v.getVariantName().equals(p.getVariantName()))
                        .findFirst();

                    if(match.isPresent()) {

match.get().setPerfumeId(p.getPerfume().getId());
                        trackVariant(match.get());
                        successCount++;
                    } else {
                        throw new RuntimeException("Variant " +
p.getVariantName() + " no longer directly found on page.");
                    }
                }

                // Slight delay to avoid hammering servers too
aggressively
                Thread.sleep(1000);
            } catch (Exception e) {
```

```
                errorCount++;
                log.error("Failed to update price for ID {}:
{}", p.getId(), e.getMessage());
            }
        }

        log.info("Finished price update. Success: {}, Errors:
{}", successCount, errorCount);
    }

    @Transactional
    public PerfumePriceDto saveManualPrice(PerfumePriceDto dto)
{
        Perfume perfume =
perfumeRepository.findById(dto.getPerfumeId())
                .orElseThrow(() -> new
IllegalArgumentException("Perfume not found"));

        Store store =
storeRepository.findByName(dto.getStoreName())
                .orElseGet(() -> {
                    Store newStore = new Store();
                    newStore.setName(dto.getStoreName());
                    newStore.setParserType("Manual");
                    return storeRepository.save(newStore);
                });

        Optional<PerfumePrice> existingPriceOpt =
perfumePriceRepository.findAll().stream()
                .filter(p ->
p.getPerfume().getId().equals(perfume.getId())
```

```
                        &&
p.getStore().getId().equals(store.getId())
                        && p.getVariantName() != null &&
p.getVariantName().equals(dto.getVariantName()))
                .findFirst();

        PerfumePrice perfumePrice;
        if (existingPriceOpt.isPresent()) {
            perfumePrice = existingPriceOpt.get();
            if (perfumePrice.getPrice() != null &&
perfumePrice.getPrice().compareTo(dto.getPrice()) != 0) {
                PriceHistory history = new PriceHistory();
                history.setPrice(perfumePrice.getPrice());

history.setCurrency(perfumePrice.getCurrency());
                history.setTimestamp(LocalDateTime.now());
                history.setPerfumePrice(perfumePrice);
                perfumePrice.getHistory().add(history);
            }
            perfumePrice.setPrice(dto.getPrice());
            perfumePrice.setCurrency(dto.getCurrency());
            if (dto.getProductUrl() != null &&
!dto.getProductUrl().isEmpty()) {

perfumePrice.setProductUrl(dto.getProductUrl());
            }
        } else {
            perfumePrice = new PerfumePrice();
            perfumePrice.setPerfume(perfume);
            perfumePrice.setStore(store);
            perfumePrice.setVolume(dto.getVolume());
```

```
            perfumePrice.setVariantName(dto.getVariantName());
            perfumePrice.setProductUrl(dto.getProductUrl());
            perfumePrice.setPrice(dto.getPrice());
            perfumePrice.setCurrency(dto.getCurrency());

            PriceHistory history = new PriceHistory();
            history.setPrice(dto.getPrice());
            history.setCurrency(dto.getCurrency());
            history.setTimestamp(LocalDateTime.now());
            history.setPerfumePrice(perfumePrice);
            perfumePrice.getHistory().add(history);
        }

        PerfumePrice saved =
perfumePriceRepository.save(perfumePrice);
        dto.setId(saved.getId());
        return dto;
    }

    @Transactional
    public void deletePrice(Long id) {
        perfumePriceRepository.deleteById(id);
    }
}

package com.aggregator.service;

import com.aggregator.dto.StoreDto;
import com.aggregator.entity.Store;
import com.aggregator.repository.StoreRepository;
import org.springframework.beans.factory.annotation.Autowired;
```

```java
import org.springframework.stereotype.Service;
import
org.springframework.transaction.annotation.Transactional;

import java.util.List;
import java.util.stream.Collectors;

@Service
public class StoreService {

    private final StoreRepository storeRepository;

    @Autowired
    public StoreService(StoreRepository storeRepository) {
        this.storeRepository = storeRepository;
    }

    @Transactional(readOnly = true)
    public List<StoreDto> getAllStores() {
        return storeRepository.findAll().stream()
                .map(this::mapToDto)
                .collect(Collectors.toList());
    }

    @Transactional
    public StoreDto createStore(StoreDto dto) {
        Store store = new Store();
        store.setName(dto.getName());
        store.setWebsite(dto.getWebsite());
        store.setParserType(dto.getParserType());

        Store saved = storeRepository.save(store);
```

```
        return mapToDto(saved);
    }

    private StoreDto mapToDto(Store store) {
        StoreDto dto = new StoreDto();
        dto.setId(store.getId());
        dto.setName(store.getName());
        dto.setWebsite(store.getWebsite());
        dto.setParserType(store.getParserType());
        return dto;
    }
}
```

/util - допоміжні файли, наразі це утиліта яка дозволяє за допомогою регулярних виразів отримувати ціну та об’єм

```
package com.aggregator.util;

import java.math.BigDecimal;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class PriceExtractor {

    // Matches digits, optional spaces, and decimal separators
(, or .)
    private static final Pattern PRICE_PATTERN =
Pattern.compile("(\\d+[\\d\\s]*[.,]?\\d*)");

    public static BigDecimal extract(String text) {
        if (text == null || text.trim().isEmpty()) {
```

```
            return null;
        }

        Matcher matcher = PRICE_PATTERN.matcher(text);
        if (matcher.find()) {
            String match = matcher.group(1);
            // Remove spaces
            match = match.replaceAll("\\s+", "");
            // Replace comma with dot for BigDecimal parsing
            match = match.replace(',', '.');
            try {
                return new BigDecimal(match);
            } catch (NumberFormatException e) {
                return null;
            }
        }
        return null;
    }

    public static Integer extractVolume(String text) {
        if (text == null) return null;
        java.util.regex.Matcher m =
java.util.regex.Pattern.compile("(?i)(\\d+)\\s*(ml|мл|oz)").mat
cher(text);
        if (m.find()) {
            try {
                return Integer.parseInt(m.group(1));
            } catch(Exception e){}
        }
        // Fallback if it's just a number
        try {
```

```
            String digits = text.replaceAll("\\D", "");
            if (!digits.isEmpty() && digits.length() <= 4) {
                return Integer.parseInt(digits);
            }
        } catch(Exception e){}
        return null;
    }
}
```

Робота застосунку

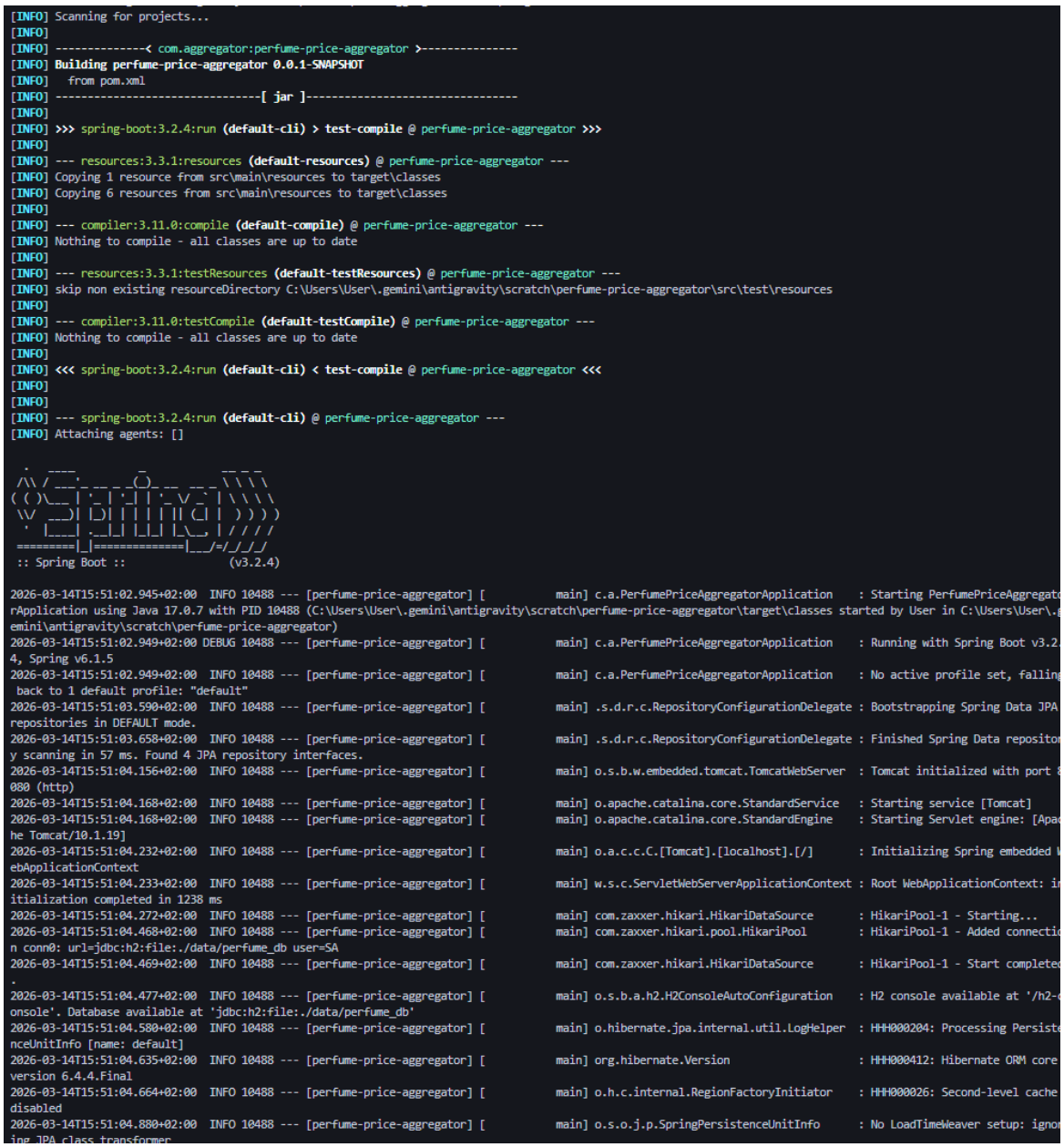


Рис.2 Запуск проэкту

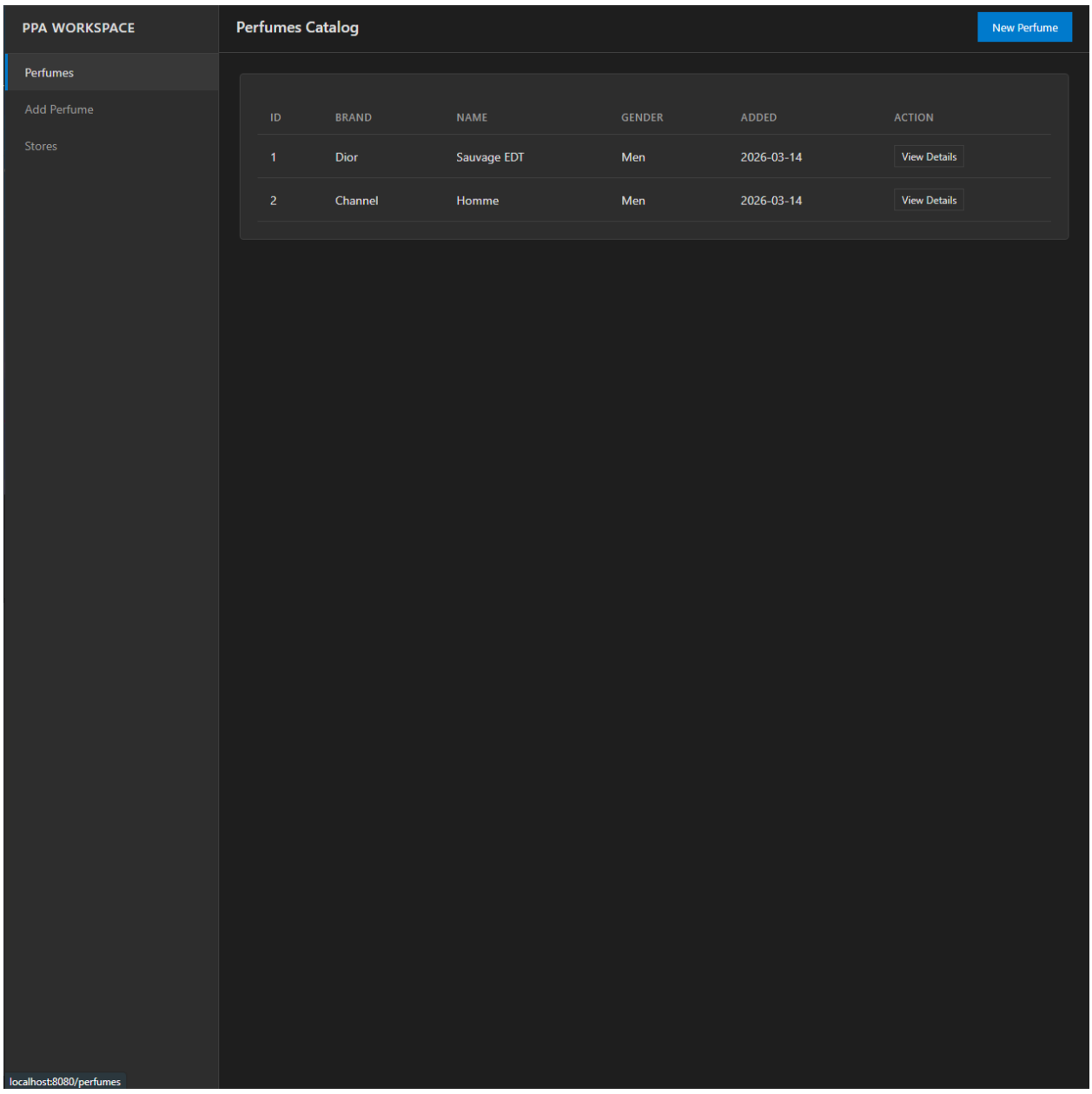


Рис.3 - Головна сторінка

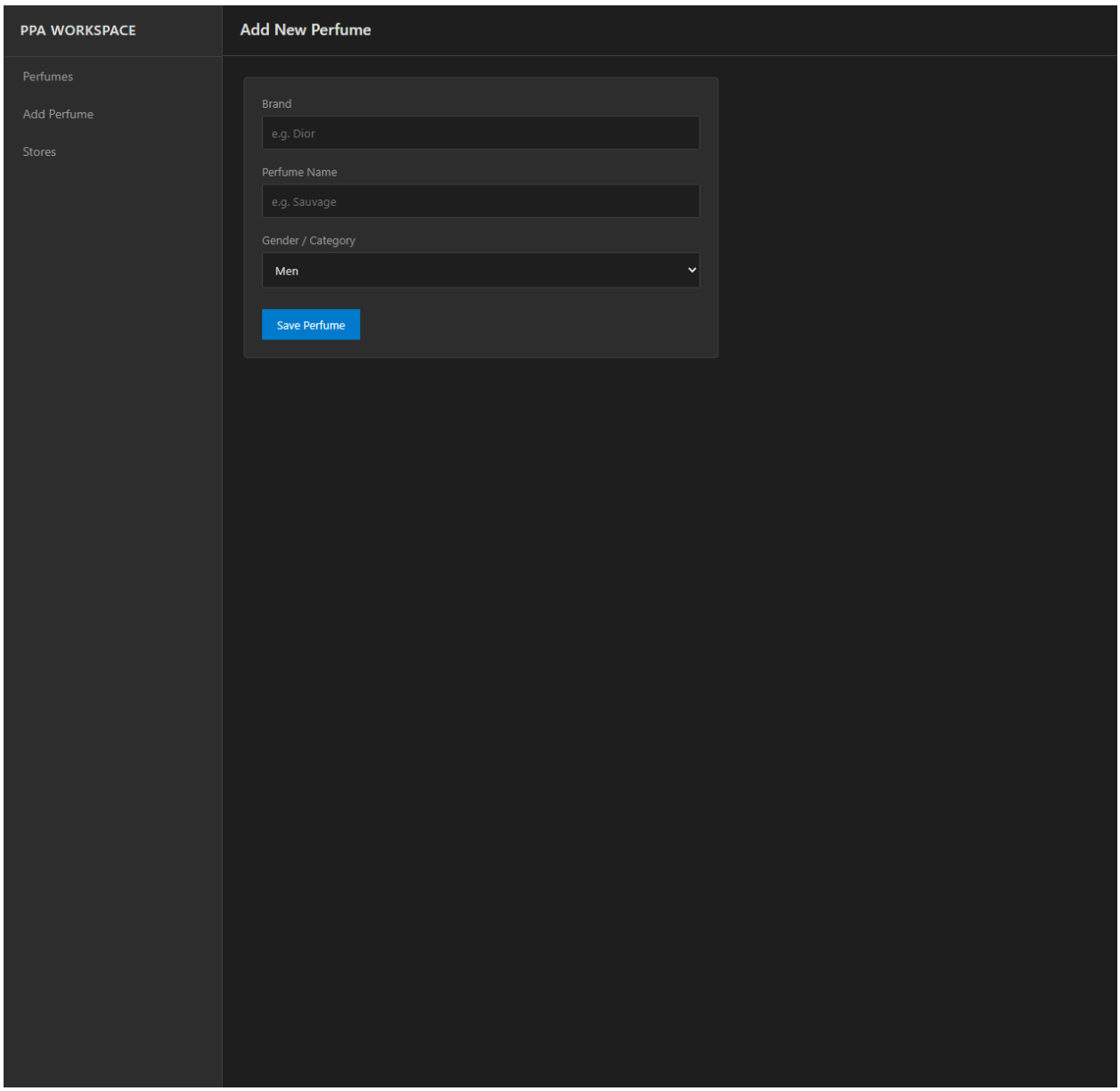
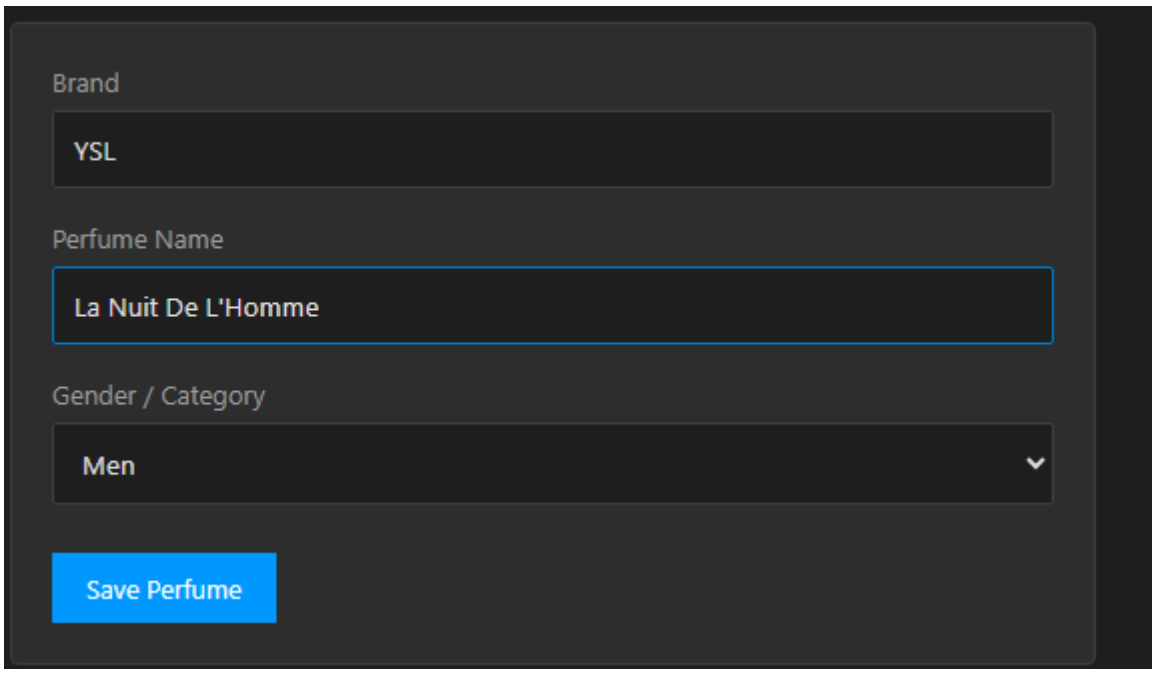


Рис 4. Вікто створення



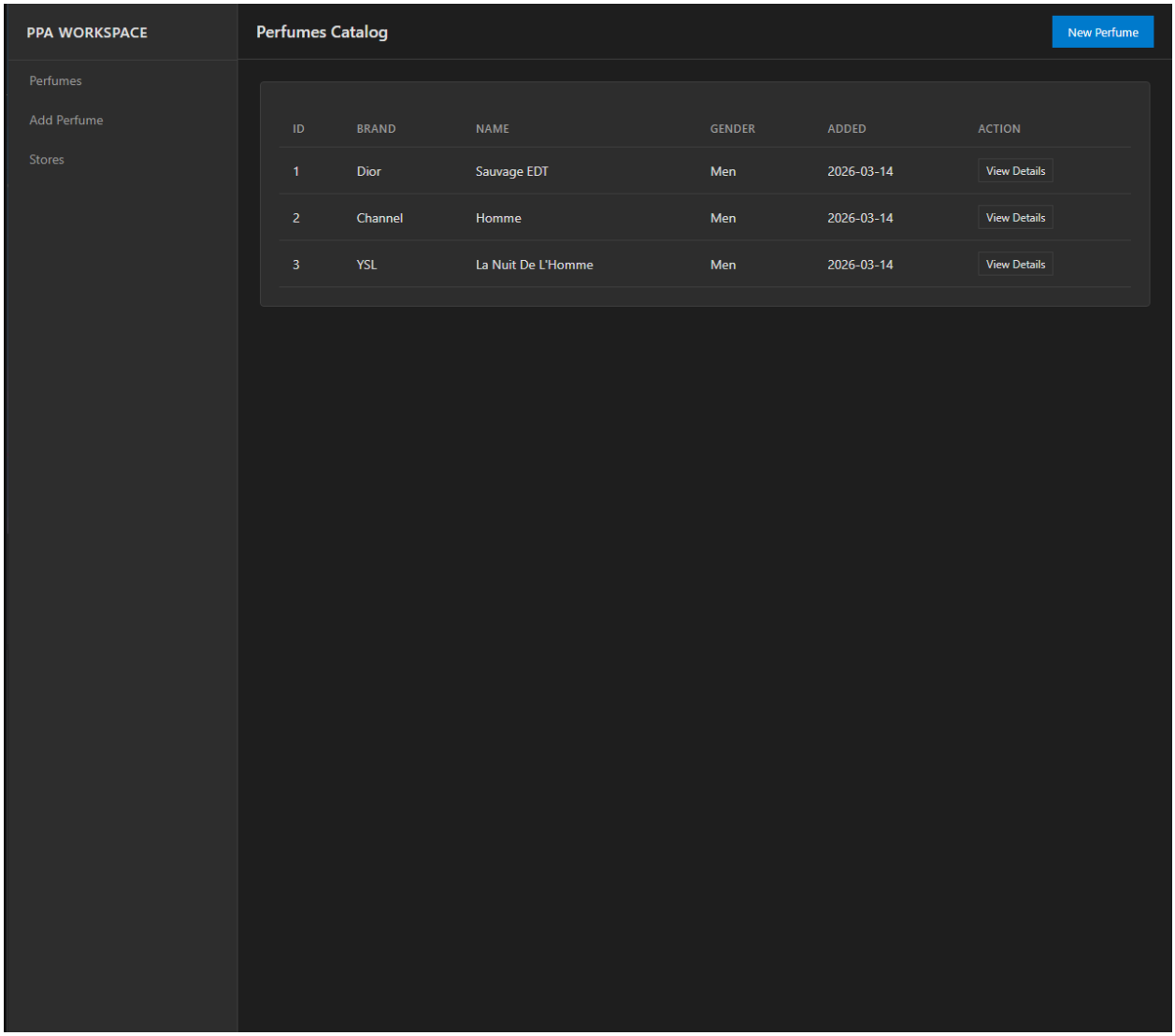


Рис.5 - Додавання нових парфумів

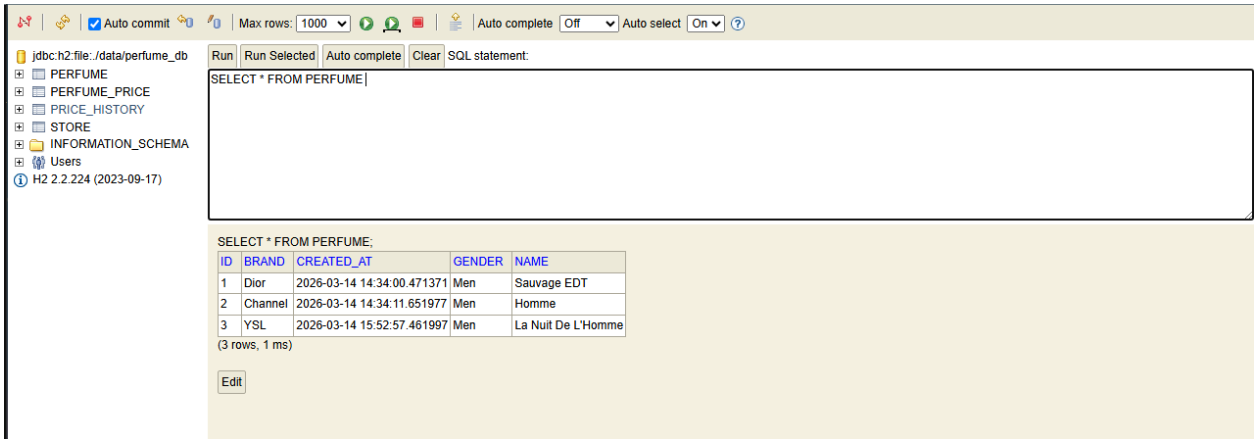


Рис.6. - Перевіряємо додавання в БД

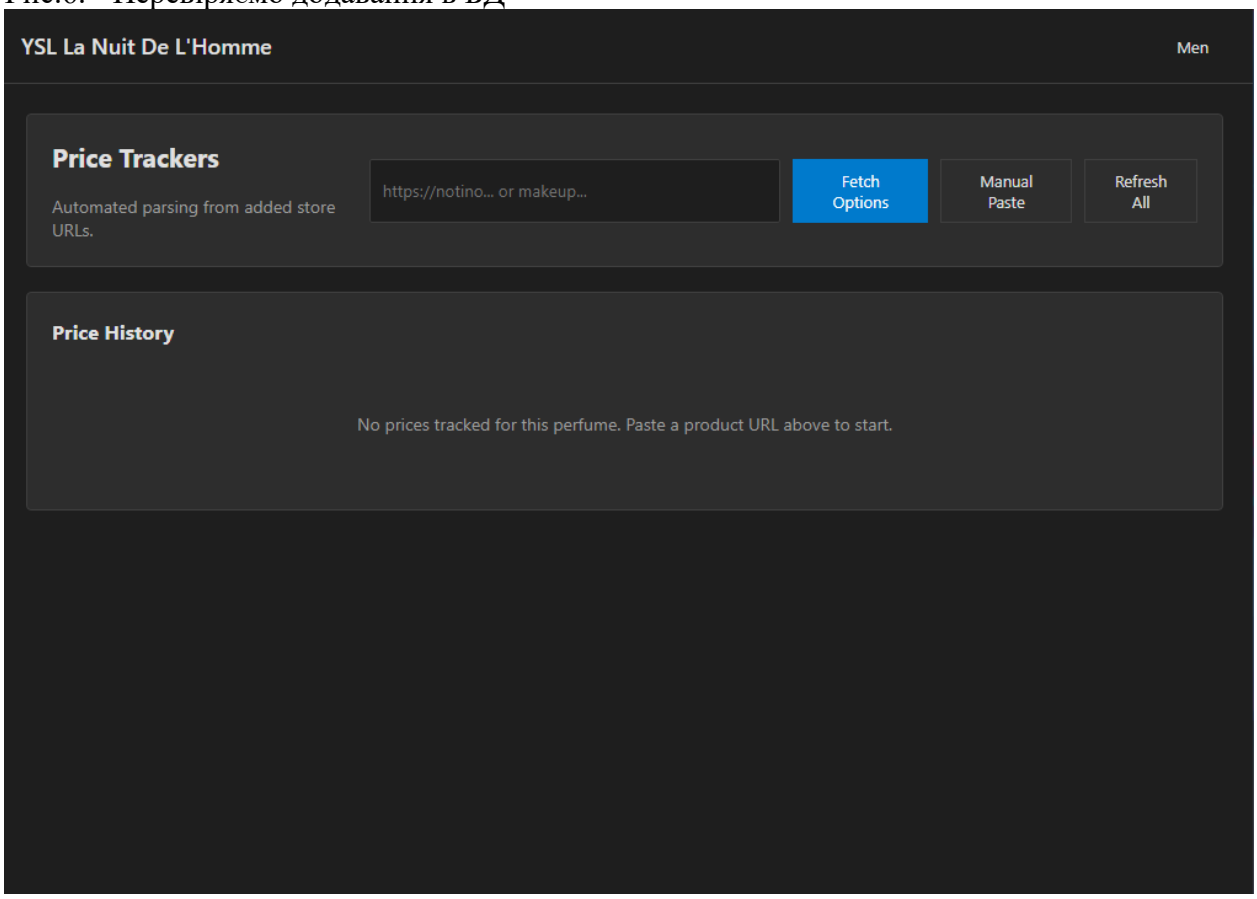


Рис.7 - Вікно додавання магазину

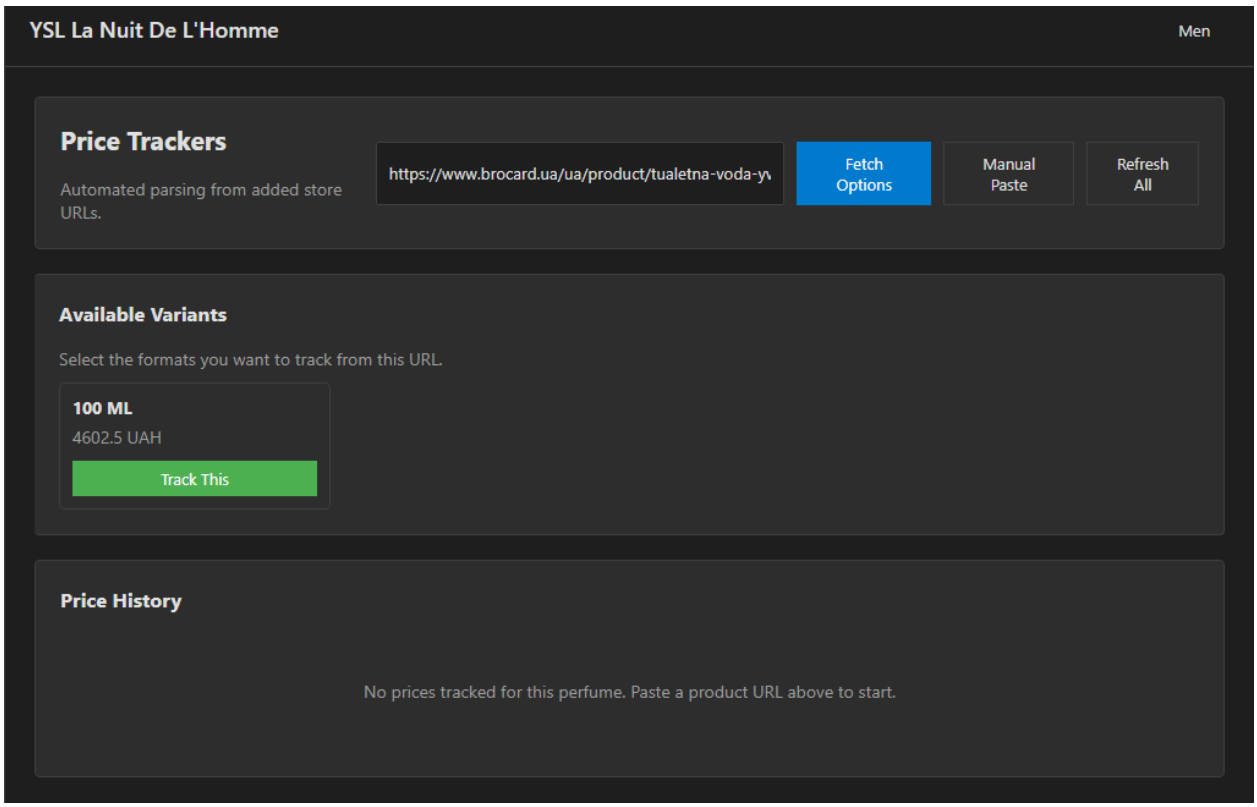


Рис. 8 - Отримуємо доступні опції по об’єму

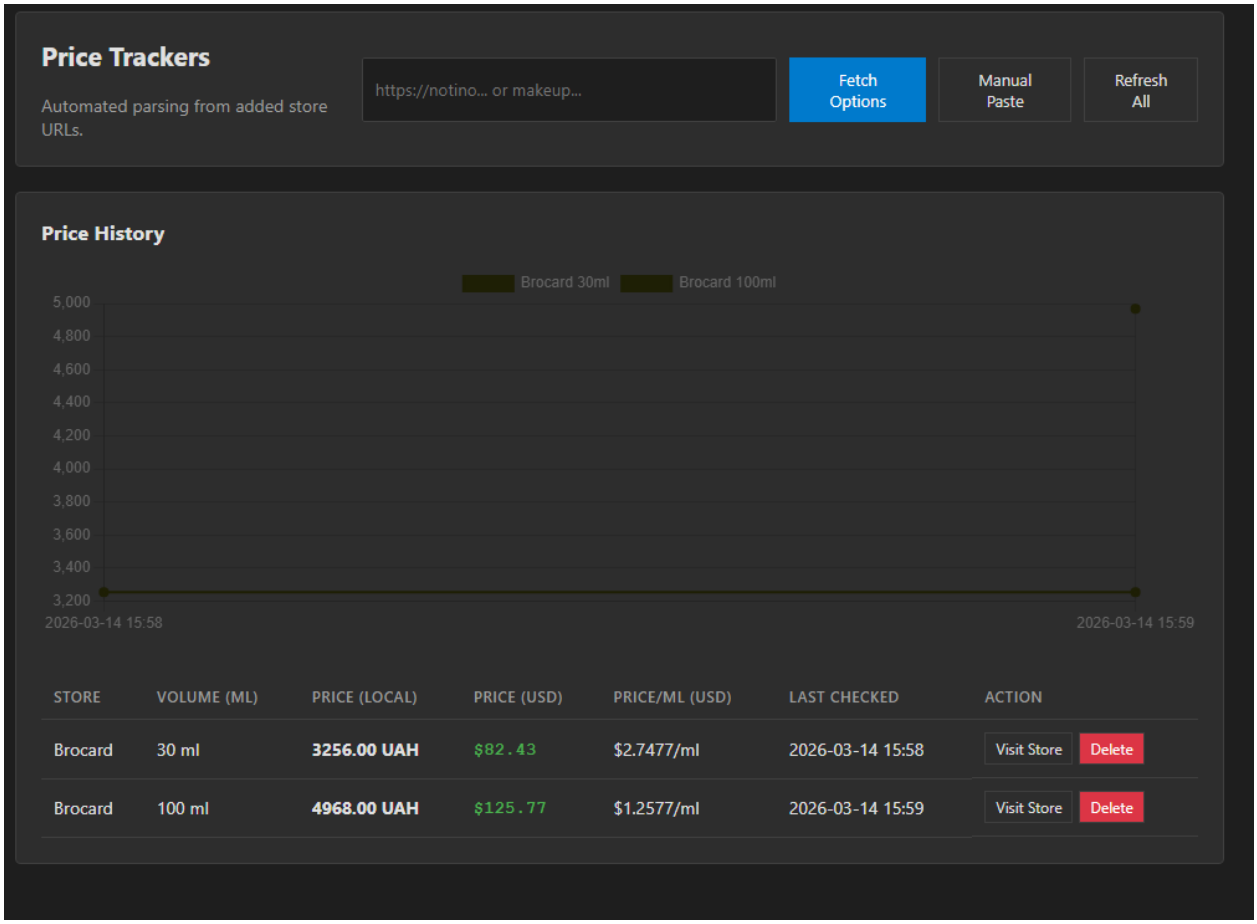


Рис 9 - інтерфейс після додавання